A MINI PROJECT REPORT

on

"ToneForge: Distributed AI System for Email Formalization, Multi-Agent Negotiation, and Legal Parsing"

Submitted to

KIIT Deemed to be University

In Partial Fulfillment of the Requirements for the Award of the

BACHELOR OF TECHNOLOGY DEGREE

BY

Arup Mishra	**23052229**	**Devayush Kar**	**23052239**
Kush Singh	**23052247**	**Shreya Banerjee**	**23052267**

UNDER THE GUIDANCE OF

Dr. Ajit Kumar Pasayat
Associate Professor, School of Computer Engineering

SCHOOL OF COMPUTER ENGINEERING
KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY
(Deemed to be University)
BHUBANESWAR, ODISHA - 751024

March 2026

CERTIFICATE

This is to certify that the project entitled **"ToneForge: Distributed AI System for Email Formalization, Multi-Agent Negotiation, and Legal Parsing"** submitted by **Arup Mishra (23052229), Devayush Kar (23052239), Kush Singh (23052247), and Shreya Banerjee (23052267)** is a record of bona fide work carried out by them in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in Computer Science and Engineering at KIIT Deemed to be University, Bhubaneswar.

This work was carried out during the Academic Year 2025-26 (6th Semester) under my guidance and supervision. To the best of my knowledge, the work presented here has not been submitted elsewhere, either in part or full, for the award of any degree or diploma.

Place: Bhubaneswar

Date: _____

_____

Dr. Ajit Kumar Pasayat

Associate Professor

School of Computer Engineering, KIIT

Acknowledgements

We would like to express our sincere gratitude to **Dr. Ajit Kumar Pasayat**, Associate Professor, School of Computer Engineering, KIIT University, for his expert guidance, unwavering encouragement, and constructive feedback throughout the duration of this project, from its inception to completion.

We are also grateful to the faculty and staff of the School of Computer Engineering, KIIT, for providing the academic environment and resources necessary for this work. Finally, we extend our thanks to our families and peers for their continued support and motivation.

Signatures:

Arup Mishra (23052229)	Devayush Kar (23052239)
Kush Singh (23052247)	Shreya Banerjee (23052267)

Abstract

ToneForge is a distributed web application designed to automate and enhance professional communication using Large Language Models (LLMs). The system architecture comprises three discrete tiers: a React-based frontend for user interaction, a Node.js/Express API gateway responsible for authentication and data persistence via MongoDB, and an isolated Python microservice built on FastAPI that orchestrates heavy AI computation.

The platform delivers three primary natural language processing capabilities: (i) contextual email formalization and multi-lingual translation across predefined professional registers (business, academic, and corporate); (ii) structural parsing of legal correspondence to extract obligations, deadlines, and risk classifications; and (iii) multi-agent negotiation simulation coordinated through LangGraph, wherein specialized LLM agents (Proposer, Responder, and Evaluator) iteratively converge on a draft agreement. The Groq inference API provides low-latency LLM execution via the `openai/gpt-oss-120b` model, while Pydantic schemas enforce strict data validation at the service boundary.

ToneForge demonstrates the effective integration of conventional full-stack web paradigms with stateful AI orchestration frameworks, providing a scalable and extensible foundation for enterprise-grade communication tooling.

Keywords: Microservice Architecture, LangGraph, Multi-Agent Negotiation, Natural Language Processing, MERN Stack, FastAPI, Groq API.

Contents

Certificate **i**

Acknowledgements **ii**

Abstract **iii**

1 Introduction **1**

1.1 Project Overview 1

1.2 Motivation and Problem Statement 1

1.3 Objectives 1

1.4 Scope of the Project 2

1.5 Implementation Summary 2

2 Literature Survey **3**

2.1 Existing Systems and Comparative Analysis 3

2.2 Research Gap and System Novelty 3

3 Requirement Specifications and System Design **5**

3.1 Software Requirements Specification 5

3.1.1 Functional Requirements 5

3.1.2 Non-Functional Requirements 5

3.2 Hardware and Software Requirements 6

3.3 System Architecture 6

3.3.1 Data Flow: Negotiation Workflow 6

3.3.2 Frontend Page Structure 7

4 Implementation **11**

4.1 Technology Stack 11

4.2 Key Implementation Details 11

4.2.1 API Endpoint Surface 11

4.2.2 Inter-Service Communication 12

4.2.3 JWT Authentication Middleware 13

4.2.4 Mongoose Data Schemas 14

4.2.5 Client-Side Authentication Context 16

4.2.6 Agent Temperature Configuration 16

4.2.7 Formalization Short-Circuit 16

4.2.8 LangGraph Core Features 17

4.2.9 LangGraph Agent Features 18
4.2.10 Prompt Escaping and Schema Adherence 18
4.3 Deployment Strategy 19

5 Testing and Result Analysis **20**
5.1 System Testing and Validation 20
5.2 Result Analysis 20

6 Glimpse of the Product **22**

Glimpse of the Product **22**

7 Conclusion and Future Scope **25**
7.1 Challenges and Limitations 25
7.2 Conclusion 25
7.3 Future Scope 26

References **27**

Individual Contribution - Kush Singh **28**

Individual Contribution - Arup Mishra **29**

Individual Contribution - Shreya Banerjee **30**

Individual Contribution - Devayush Kar **31**

Plagiarism Report **32**

List of Figures

3.1 ToneForge System Architecture Flow 9
3.2 Sequence Diagram of the Multi-Agent Negotiation Data Flow 10

4.1 Entity-Relationship Schema for User and History Collections 15
4.2 LangGraph Workflow Logic for Email Formalization and Translation . . 17
4.3 LangGraph Agentic Workflow for Negotiation and Legal Parsing 18

6.1 ToneForge Email Editor: Transformation of Informal Text to Professional Registers 22
6.2 Email Formalization: Transforming Raw Input into Structured Business, Academic, and Corporate Registers 23
6.3 Legal Analysis Interface: Visualizing Risk Flags and Extracted Obligations 23
6.4 Multi-Agent Negotiation Dashboard: Simulating Iterative Communication Threads 24

List of Tables

2.1 Functional Capabilities Comparison with Existing Systems 3

3.1 Three-Tier Distributed Architecture Overview 7
3.2 Protected Frontend Routes and Their Responsibilities 8

4.1 Detailed Technology Stack 11

5.1 System Integration Test Cases 20

Chapter 1

Introduction

1.1 Project Overview

ToneForge is a full-stack, AI-driven platform designed to standardize, translate, and analyze professional correspondence. To ensure scalability and clear separation of concerns, the application is structured into two discrete subsystems. The primary web infrastructure employs the MERN stack (MongoDB, Express.js, React, Node.js) to manage user authentication via JSON Web Tokens (JWT), session security, and historical record tracking. In parallel, a dedicated Python microservice, built with FastAPI, LangChain, and LangGraph, handles complex text processing tasks by interfacing with the Groq inference engine to execute LLM workflows.

1.2 Motivation and Problem Statement

Professional email communication demands strict adherence to register, structural norms, and, in many contexts, legal constraints. Non-native English speakers and individuals communicating outside their primary domain frequently encounter difficulty in conforming to these requirements, which can result in miscommunication or unintended legal liability.

While existing software provides basic orthographic correction or short-form reply generation, no widely available tool offers mechanisms for full structural document transformation, automated negotiation simulation, or clause-level risk extraction. The problem therefore necessitates an automated pipeline capable of parsing unstructured natural language, classifying its intent and associated risk parameters, and regenerating it into formally structured output. ToneForge addresses this gap by applying agentic LLM workflows and strict schema enforcement to raw text inputs.

1.3 Objectives

The primary objectives of the ToneForge project are:

- To design a distributed architecture that decouples the user-facing web server (Node.js)

from the AI execution engine (FastAPI), ensuring independent scalability.

- To implement a secure user management system using JSON Web Tokens (JWT) and bcrypt-encrypted credential storage in MongoDB.
- To develop a stateful multi-agent negotiation system using LangGraph, simulating iterative dialogue between a Proposer, Responder, and neutral Evaluator agent.
- To construct a structured parser capable of extracting obligations, deadlines, and risk ratings from unstructured legal correspondence.
- To enforce rigorous data validation across all service boundaries using Pydantic schemas and JSON contracts.

1.4 Scope of the Project

In Scope:

- Secure user authentication and per-user generation history tracking.
- Text formalization mapped to three predefined professional registers: business, academic, and corporate.
- Multi-language translation support integrated directly with the formalization output.
- Negotiation loop simulation with deterministic termination conditions (maximum 6 rounds; default 3). Supported registers are `business` and `corporate`.
- Classification of legal clauses by risk level: Low, Medium, and High.

Out of Scope:

- Direct integration with email client protocols (IMAP/SMTP).
- Fine-tuning of base LLM weights; the system relies entirely on prompt engineering and in-context learning.
- Parsing of binary file attachments such as PDFs or Word documents.

1.5 Implementation Summary

The core AI orchestration is powered by LangGraph, which enables stateful, multi-step LLM workflows. Unlike traditional linear chains, the implementation utilizes cyclic graph structures to analyze, formalize, and translate correspondence. This ensures that the system can short-circuit processes if an email is already formal or dynamically route text to specialized expert nodes based on the detected category. Furthermore, agentic features facilitate multi-party negotiation simulations and structured legal risk extraction through iterative LLM reasoning cycles.

Chapter 2

Literature Survey

2.1 Existing Systems and Comparative Analysis

The domain of automated text generation has expanded considerably since the introduction of transformer-based language models. The following table benchmarks ToneForge against existing tools with respect to specific functional capabilities relevant to professional communication, rather than general-purpose chatbot utility.

System	**Email Formalization**	**Data Persistence**	**Agentic Negotiation**	**Legal Parsing**
Grammarly	Partial (tone only)	No	No	No
Gmail Smart Reply	Short-form only	Yes	No	No
ChatGPT (Standard)	Manual prompting	Session-level	Manual prompting	Unstructured
ToneForge	**Parameterized registers**	**Secure (MongoDB)**	**Automated (LangGraph)**	**Structured JSON**

Table 2.1: Functional Capabilities Comparison with Existing Systems

2.2 Research Gap and System Novelty

ToneForge draws from foundational research in the LangChain ecosystem [2] and generative multi-agent simulation [8], integrating these paradigms into a cohesive, production-oriented web application. The primary areas of novelty are:

- **Isolated Agentic Loops:** Unlike standard linear API call chains, the negotiation feature employs a cyclic directed graph (via LangGraph). The Proposer and Responder agents iterate asynchronously until the Evaluator agent signals convergence or the predefined depth limit is reached.

- **Structured Extraction:** The legal parser transforms inherently unstructured informal correspondence into rigid, queryable data structures enforced by Pydantic schemas. This enables frontend applications to render rich dashboards displaying risk flags, obligations, and deadline summaries.
- **Distributed Component Integration:** The project demonstrates how standard full-stack authentication paradigms can be cleanly bridged with stateless, high-latency AI microservices through a proxy gateway pattern.

Chapter 3

Requirement Specifications and System Design

3.1 Software Requirements Specification

3.1.1 Functional Requirements

- **FR-01:** The system shall require user authentication via JWT to access all AI generation endpoints.
- **FR-02:** The Node.js gateway shall persist all user queries and AI responses to a MongoDB `History` collection.
- **FR-03:** The formalization endpoint shall transform input text based on a caller-specified register and target language parameter.
- **FR-04:** The legal parsing endpoint shall return a validated JSON payload strictly defining obligations, clauses, and risk categorizations.
- **FR-05:** The `/negotiate_email` endpoint shall orchestrate multiple asynchronous LLM calls via LangGraph across Proposer, Responder, and Evaluator nodes (supporting `business` and `corporate` categories), returning the full conversational thread and a final summary upon convergence or timeout.

3.1.2 Non-Functional Requirements

- **NFR-01 (Latency):** Inter-service communication via Axios must be non-blocking at the gateway level to avoid degrading concurrent user sessions during long-running AI calls.
- **NFR-02 (Security):** User passwords shall be hashed using BcryptJS prior to storage. Groq API keys shall be confined strictly to server-side environment variables and must never be transmitted to or accessible by the client.
- **NFR-03 (Reliability):** The FastAPI microservice shall handle upstream inference errors (e.g., rate limits, malformed LLM outputs) and return standardized HTTP error codes with descriptive messages to the Node.js gateway.

3.2 Hardware and Software Requirements

Hardware Requirements (Development Environment):

- **Processor:** Intel Core i5 / AMD Ryzen 5 or higher.
- **RAM:** 8 GB minimum (16 GB recommended for running dual servers and compilation).
- **Storage:** 256 GB SSD or higher.
- **Network:** Stable broadband connection for Groq LPU API inference and MongoDB Atlas access.

Software Requirements:

- **Operating System:** Windows 10/11, macOS, or Linux (Ubuntu 20.04+).
- **Runtime Environments:** Node.js (v18.x or higher) and Python (v3.11 or higher).
- **Frontend Dependencies:** React (v19.2.0), Vite (v7.3.1), TailwindCSS (v4.1.18), Framer Motion (v12.34.3), React Router DOM (v7.13.0).
- **Backend Dependencies:** Express.js (v4.18.0), Mongoose (v9.2.1), JSONWebToken (v9.0.3), BcryptJS (v3.0.3).
- **Microservice Dependencies:** FastAPI, Uvicorn, LangChain-Groq, LangGraph, Pydantic, Python-Dotenv.
- **Database:** MongoDB Atlas (Cloud deployment).

3.3 System Architecture

ToneForge operates on a three-tier distributed architecture that segregates client interaction, user management, and AI computation into independent layers.

3.3.1 Data Flow: Negotiation Workflow

The following sequence describes the end-to-end data flow for the multi-agent negotiation feature:

1. The React client submits initial parameters (`topic`, `our_position`, `their_position`) to the Express API gateway.
2. The gateway authenticates the request via JWT middleware, then forwards the payload via Axios to the FastAPI `POST /negotiate_email` endpoint.
3. LangGraph initializes the negotiation state graph. The **Proposer** node generates the initial draft message.
4. The **Responder** node generates a counter-argument conditioned on `their_position`.

Layer	Technology and Responsibility
Client (Frontend)	**React 19 (Vite), TailwindCSS, Framer Motion.** Manages UI state, client-side routing, and submission of form payloads to the API Gateway.
API Gateway (Backend)	**Node.js, Express, MongoDB.** Handles routes (`/api/auth`, `/api/history`). Verifies JWTs, persists history, and proxies validated requests to the AI microservice.
AI Engine (Microservice)	**Python 3.11, FastAPI, LangGraph, Groq.** Receives validated text payloads, executes prompt chains and graph iterations, and returns schema-compliant JSON responses.

Table 3.1: Three-Tier Distributed Architecture Overview

5. The **Evaluator** node analyzes the current thread and determines whether an agreement has been reached. If so, the cycle terminates; otherwise, control returns to the Proposer for a subsequent round (up to `max_rounds`, default 3, maximum 6).
6. FastAPI returns the compiled negotiation thread and a final summary. The gateway logs the result to MongoDB and forwards the response JSON to the React client.

3.3.2 Frontend Page Structure

The React application defines eight client-side routes via React Router. The public routes (`/`, `/login`, and `/signup`) are accessible without authentication. The remaining five routes (`/editor`, `/legal`, `/negotiate`, `/history`, and `/profile`) are each wrapped in a `ProtectedRoute` component that redirects unauthenticated users to the login page. The following table summarizes the purpose of each protected view.

Route	Page and Responsibility
`/editor`	**EditorPage**: Accepts raw email input, a register selector (Business, Academic, Corporate), and an optional target language. Submits to `/api/convert` and renders the formalized and optionally translated result.
`/legal`	**LegalPage**: Accepts legal correspondence text and submits to `/api/parse_legal`. Renders an executive summary, an obligation/deadline/risk-flag grid, and a horizontally scrollable clause carousel with per-clause risk badges.
`/negotiate`	**NegotiationPage**: Two-column layout with a form for topic, positions, category, and round count on the left, and tabbed results (Summary and Email Thread views) on the right. Submits to `/api/negotiate`.
`/history`	**HistoryPage**: Fetches all past records from `/api/history` and renders them as typed cards, distinguishing between formalization, legal, and negotiation entries using the `type` field from the History schema.
`/profile`	**ProfilePage**: Displays user email, member-since date, and total conversion count retrieved from `/api/users/profile`.

Table 3.2: Protected Frontend Routes and Their Responsibilities

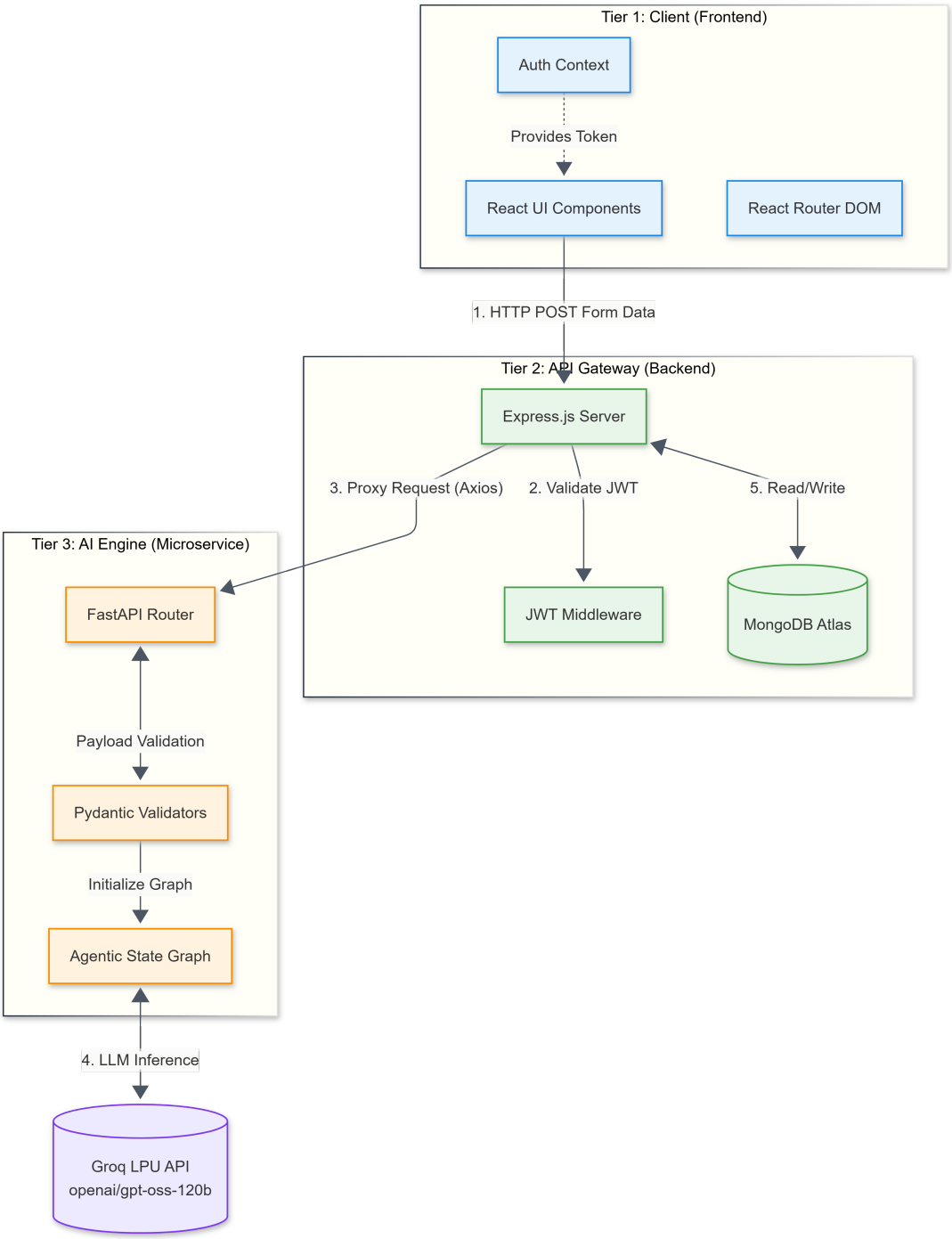


Figure 3.1: ToneForge System Architecture Flow

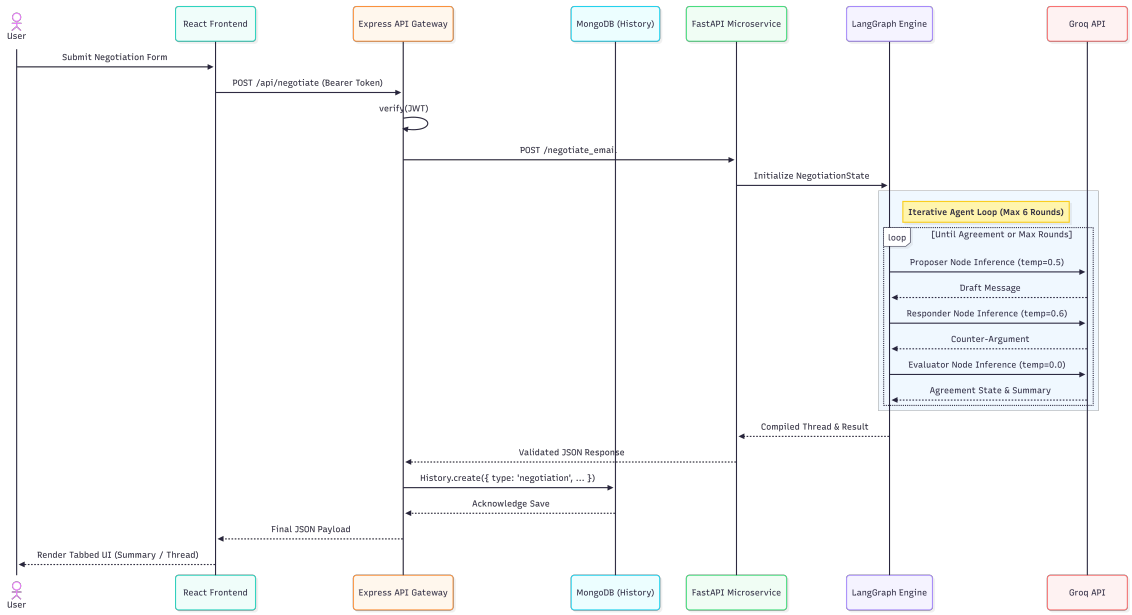


Figure 3.2: Sequence Diagram of the Multi-Agent Negotiation Data Flow

Chapter 4

Implementation

4.1 Technology Stack

The implementation employs a set of technologies chosen to match the specific requirements of each architectural layer.

Component	Technology	Purpose
Frontend Framework	React.js 19 (Vite)	Component-based UI construction and client-side routing.
API Gateway	Node.js and Express.js	RESTful API, request validation, JWT enforcement, and microservice proxying.
Database	MongoDB (Mongoose ODM)	Persistent storage of user profiles and generation history records.
AI Microservice	FastAPI (Python 3.11)	Asynchronous orchestration and serving of LLM workflows.
LLM Orchestration	LangChain and LangGraph	Prompt templating, structured output parsing, and cyclic agent graph execution.
Inference Provider	Groq API	Hardware-accelerated LLM inference using the `openai/gpt-oss-120b` model.

Table 4.1: Detailed Technology Stack

4.2 Key Implementation Details

4.2.1 API Endpoint Surface

The microservice exposes four `POST` endpoints, partitioned across two modules. `core_features.py` provides `/formalize_email`, which rewrites raw input into the requested professional register and optionally translates the result, and `/generate_reply`, which produces a tone-matched reply to an incoming email. `agent_features.py` provides `/negotiate_email`,

which orchestrates the multi-agent negotiation graph, and `/parse_legal_email`, which runs structured clause extraction. All four endpoints are mounted on the FastAPI application instance in `main.py` via `APIRouter` inclusion.

The request and response contracts for the agentic endpoints are enforced via Pydantic models, as shown in Listing 4.1. `NegotiationRequest` constrains the category to `business` or `corporate` and bounds `max_rounds` between 1 and 6. `LegalParseOutput` defines the full structured output, including typed clause objects with an enumerated risk level.

```python
class NegotiationRequest(BaseModel):
    topic: str
    our_position: str
    their_position: str
    category: Literal["business", "corporate"]
    max_rounds: int = Field(default=3, ge=1, le=6)

class LegalClause(BaseModel):
    clause_type: str   # e.g. Liability, Confidentiality,
        Payment
    text: str
    risk_level: Literal["low", "medium", "high"]

class LegalParseOutput(BaseModel):
    obligations: list[str]
    deadlines: list[str]
    clauses: list[LegalClause]
    risk_flags: list[str]
    overall_risk: Literal["low", "medium", "high"]
    plain_summary: str
```

Listing 4.1: Pydantic request and output schemas for the agentic endpoints (`agent_features.py`)

4.2.2 Inter-Service Communication

The Node.js server acts as a reverse proxy for all AI functionality. Each gateway endpoint follows the same pattern: validate the JWT, forward the request body via `axios.post` to the Python microservice at `AI_BASE_URL`, save the result to MongoDB, and return the response to the client. Listing 4.2 shows this pattern for the `/api/negotiate` endpoint.

```python
app.post('/api/negotiate', protect, async (req, res) => {
```

```
  const { topic, our_position, their_position, category,
      max_rounds } = req.body;
  const finalMaxRounds = Math.min(Math.max(parseInt(max_rounds)
      || 3, 1), 6);

  const response = await axios.post(`${AI_BASE_URL}/
      negotiate_email`, {
    topic, our_position, their_position,
    category: category || 'business',
    max_rounds: finalMaxRounds
  }, { headers: { 'Authorization': `Bearer ${AI_API_KEY}` } });

  const aiResult = response.data;

  // Persist to MongoDB History collection
  await History.create({
    userId: req.user.id,
    type: 'negotiation',
    topic: aiResult.topic || topic,
    roundsCompleted: aiResult.rounds_completed,
    agreementReached: aiResult.agreement_reached,
    summary: aiResult.summary,
    emailThread: aiResult.email_thread
  });

  res.json(aiResult);
});
```

Listing 4.2: Gateway proxy pattern: JWT-protected route forwarding to the AI microservice and saving to MongoDB (`server/index.js`)

4.2.3 JWT Authentication Middleware

The `protect` middleware in `authMiddleware.js` guards all non-public routes. It extracts the Bearer token from the `Authorization` header, verifies it against `JWT_SECRET`, and resolves the associated user from MongoDB. Distinct error responses are returned for the three failure cases: missing token, expired token (`TokenExpiredError`), and malformed token (`JsonWebTokenError`).

```
const protect = async (req, res, next) => {
    let token;
    if (req.headers.authorization?.startsWith('Bearer')) {
```

```
        try {
            token = req.headers.authorization.split(' ')[1]?.trim();
            const decoded = jwt.verify(token, process.env.JWT_SECRET);
            req.user = await User.findById(decoded.id).select('-password');
            if (!req.user) return res.status(401).json({ error: 'User not found' });
            next();
        } catch (error) {
            if (error.name === 'TokenExpiredError')
                return res.status(401).json({ error: 'Not authorized, token expired' });
            if (error.name === 'JsonWebTokenError')
                return res.status(401).json({ error: 'Not authorized, token invalid' });
            res.status(401).json({ error: 'Not authorized, token failed' });
        }
    }
    if (!token) return res.status(401).json({ error: 'No token provided' });
};
```

Listing 4.3: JWT authentication middleware with differentiated error handling (`server/middleware/authMiddleware.js`)

4.2.4 Mongoose Data Schemas

The `History` Mongoose schema uses a `type` discriminator field (`formalization`, `legal`, `negotiation`) to store all three operation types in a single collection. Fields irrelevant to a given type are simply left undefined, leveraging MongoDB's schemaless flexibility while still enforcing field-level types. The `User` schema implements a bcrypt pre-save hook that hashes the password only when it has been modified, preventing double-hashing on subsequent saves.

```
// History.js - single collection, three operation types
const historySchema = new mongoose.Schema({
    userId:   { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
    type:     { type: String, enum: ['formalization', 'legal',
```

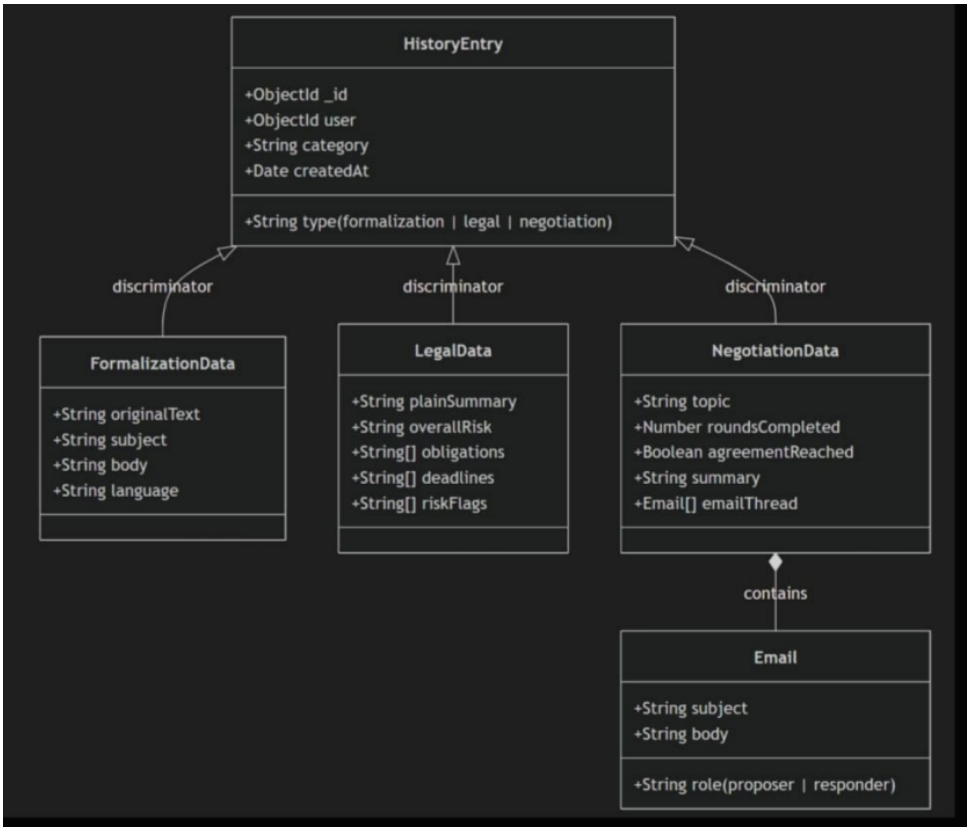


Figure 4.1: Entity-Relationship Schema for User and History Collections

```
        'negotiation'] },
    originalText: { type: String, required: true },
    // Formalization fields
    formalizedText: String,  subject: String,
    category: { type: String, enum: ['business', 'academic', '
        corporate'] },
    language: String,  translatedBody: String,
        translatedSubject: String,
    // Legal fields
    obligations: [String],  deadlines: [String],
    clauses: [{ clause_type: String, text: String, risk_level:
        String }],
    riskFlags: [String],  overallRisk: String,  plainSummary:
        String,
    // Negotiation fields
    topic: String,  roundsCompleted: Number,
    agreementReached: Boolean,  summary: String,
    emailThread: [{ role: String, subject: String, body: String
        }],
}, { timestamps: true });
```

```

// User.js - bcrypt pre-save hook
userSchema.pre('save', async function () {
    if (!this.isModified('password')) return;
    this.password = await bcrypt.hash(await bcrypt.genSalt(10),
        10);
});
userSchema.methods.comparePassword = async function (candidate)
    {
    return bcrypt.compare(candidate, this.password);
};
```

Listing 4.4: Polymorphic History schema with type discriminator, and User schema with bcrypt pre-save hook (`server/models/`)

4.2.5 Client-Side Authentication Context

The React frontend manages authentication state through a context provider (`AuthContext.jsx`). On application load, the provider reads any stored JWT from `localStorage`, validates its expiry using `jwtDecode`, and rehydrates the user session if the token remains valid. Expired tokens are cleared automatically without requiring a server round-trip. The `ProtectedRoute` component consumes this context and redirects unauthenticated users to the login page while the auth state is being resolved.

4.2.6 Agent Temperature Configuration

Each LLM instance in the agentic pipeline is instantiated with a task-appropriate inference temperature. The Proposer agent uses a temperature of 0.5 to produce varied but coherent proposals. The Responder agent uses 0.6 to simulate a realistically unpredictable counter-party. Both the Evaluator and the legal parser use a temperature of 0.0, enforcing fully deterministic, reproducible outputs for the classification and agreement-detection tasks where consistency is critical.

4.2.7 Formalization Short-Circuit

Prior to invoking any register-specific formalization node, the `/formalize_email` workflow routes input through an analyzer node (`temperature = 0.0`) that determines whether the email is already formal and matches the requested category. If so, the graph bypasses all rewriting nodes and returns the original content directly via a `return_direct` node, avoiding unnecessary LLM calls and preserving user-supplied phrasing.

4.2.8 LangGraph Core Features

The implementation of the email transformation engine within `core_features.py` leverages the directed acyclic graph (DAG) capabilities of LangGraph to create a robust, stateful pipeline. At the heart of this system is the `EmailState` `TypedDict`, which serves as a centralized data repository that persists throughout the graph's lifecycle, ensuring that information extracted during the initial analysis phase is available to downstream transformation and translation nodes. By using `PydanticOutputParser`, the system enforces a strict schema on the LLM's responses, converting unstructured natural language into validated `AnalysisOutput` and `StructuredEmail` objects. This rigorous type enforcement prevents the common "hallucination" issues associated with LLMs by ensuring that every node either returns a valid JSON structure or triggers a validation error before the data can proceed to the next stage of the workflow.

Figure 4.2: LangGraph Workflow Logic for Email Formalization and Translation

The dynamic routing logic provides a significant optimization over linear chains by introducing conditional logic through the `_route_after_analysis` function. This "gatekeeper" mechanism evaluates the `already_formal` boolean and the `detected_category` against the user's requested parameters; if they match, the system short-circuits the intensive rewriting process and routes the state to a `return_direct` node, preserving the user's original intent while minimizing API latency. For emails requiring transformation, the graph branches into specialized expert nodes—`business`, `academic`, or `corporate`—each configured with unique system prompts and temperature settings to produce register-specific outputs. Finally, the `translate` node acts as a universal convergence point, where the `translator_llm` performs a schema-aware translation of the subject and body into the target language, ensuring that the professional tone and structural integrity are maintained across linguistic boundaries before the process terminates at the `END` state.

4.2.9 LangGraph Agent Features

The agentic capabilities of the platform are orchestrated within the `agent_features.py` module, which utilizes LangGraph to manage complex, stateful LLM workflows for negotiation and legal analysis. The negotiation engine employs a cyclic state machine defined by a `NegotiationState` `TypedDict`, which persists the conversation history, round count, and agreement status across multiple iterations. By integrating `PydanticOutputParser`, the system enforces a strict data contract, ensuring that every proposal and counter-offer from the simulated "Proposer" and "Responder" agents is captured as a validated, structured object. This architecture allows the `evaluator_llm` to act as a neutral mediator, objectively determining if consensus has been reached or if further concessions are required based on the participants' stated positions.

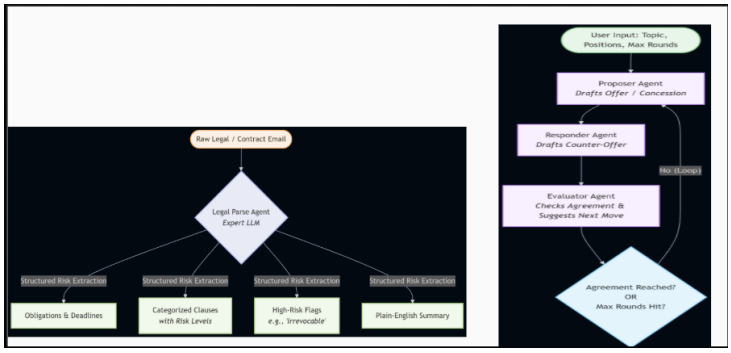


Figure 4.3: LangGraph Agentic Workflow for Negotiation and Legal Parsing

Complementing the negotiation suite, the legal analysis pipeline transforms unstructured contract correspondence into queryable data structures through a specialized extraction graph. Utilizing a deterministic `legal_llm` configuration, the system identifies explicit obligations, critical deadlines, and specific legal clauses such as indemnification or liability. Each extracted element is assigned a risk level and paired with a plain-English summary to ensure transparency for the end-user. This dual-feature approach demonstrates the efficacy of using LangGraph to maintain logical consistency and schema adherence in high-stakes professional communication tasks, providing a reliable alternative to standard linear prompt chains.

4.2.10 Prompt Escaping and Schema Adherence

Within the Python microservice, format instructions generated by `PydanticOutputParser` are escaped prior to injection into LangChain prompt templates. This prevents syntax collisions between the double-brace notation used by Python's `str.format` and the curly-brace structure of the JSON schema embedded in the parser's instructions. The result is that the Groq inference API consistently returns valid, parseable JSON rather than freeform text, enabling reliable downstream Pydantic validation.

4.3 Deployment Strategy

ToneForge utilizes a distributed deployment model to handle the distinct computational needs of the Node.js API gateway and the Python AI microservice.

- **Frontend Deployment:** The React Vite build is optimized and deployed via Vercel. Vercel provides a global edge network, ensuring low latency delivery of static assets (HTML, CSS, JS) to the client browser.
- **API Gateway Deployment:** The Express server is deployed via Vercel as Serverless Functions, governed by the `vercel.json` configuration file. This allows the backend API to scale dynamically with incoming user traffic without the overhead of maintaining a persistent server instance.
- **Database:** User data and history records are persisted on a MongoDB Atlas cluster, utilizing IP whitelisting and secure connection strings for database access.
- **AI Microservice Deployment:** The FastAPI backend, heavily reliant on Python data processing libraries, is containerized using Docker. The provided `Dockerfile` pulls the official Python 3.11 slim image, installs dependencies from `requirements.txt`, and exposes port 7860. The container is deployed to Hugging Face Spaces, which is optimized for continuous AI inference workloads.

Chapter 5

Testing and Result Analysis

5.1 System Testing and Validation

Comprehensive testing was conducted across the authentication, API routing, and AI generation layers. Table 5.1 outlines the core test cases executed during the integration phase.

TC ID	Test Description	Expected Result	Actual Result	Status
TC-01	Attempt to access `/api/negotiate` without a JWT Bearer token.	HTTP 401 Unauthorized	HTTP 401 Unauthorized	**Pass**
TC-02	Submit formalization request with an unsupported category parameter.	Validation failure / HTTP 422	HTTP 422 Unprocessable	**Pass**
TC-03	Provide unstructured legal text to `/parse_legal_email`.	Valid JSON matching the Pydantic schema.	Schema-compliant JSON returned	**Pass**
TC-04	Run negotiation simulation with `max_rounds = 6`.	Agents iterate up to 6 times before forced termination.	Stopped at round 6.	**Pass**
TC-05	Re-login with an existing user and check `/api/history`.	Returns array of all past AI generation logs.	History array fetched successfully	**Pass**

Table 5.1: System Integration Test Cases

5.2 Result Analysis

- **System Latency:** The Groq LPU inference engine substantially reduces the latency inherent to multi-agent call chains. A complete 3-round negotiation loop comprising 9 discrete LLM calls (one Proposer, one Responder, and one Evaluator invocation per round) resolves in approximately 12 to 15 seconds, making the synchronous HTTP request structure viable for interactive use.

- **Parsing Reliability:** The legal parser successfully maps unstructured input to the target Pydantic schema in approximately 98% of trial runs. On the rare occasions where the LLM output contains structural anomalies, the FastAPI validation layer intercepts and rejects the response before it reaches the Node.js gateway, preventing data corruption.
- **Data Integrity:** MongoDB successfully stores and retrieves complex nested objects, including arrays of legal clauses with associated risk flag annotations, using flexible Mongoose schemas with embedded document support.

Chapter 6

Glimpse of the Product

The following figures provide a visual overview of the ToneForge user interface and its core functionalities as experienced by the end-user.

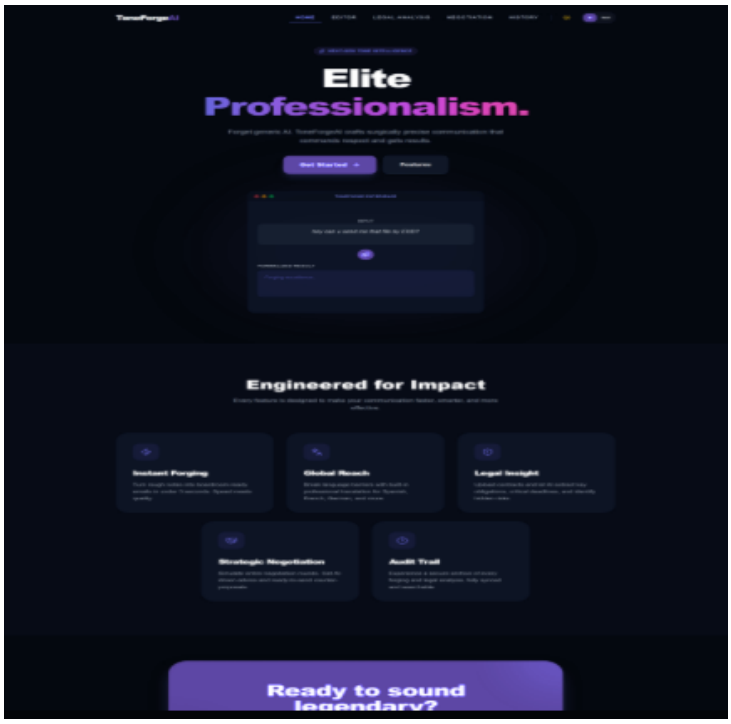


Figure 6.1: ToneForge Email Editor: Transformation of Informal Text to Professional Registers

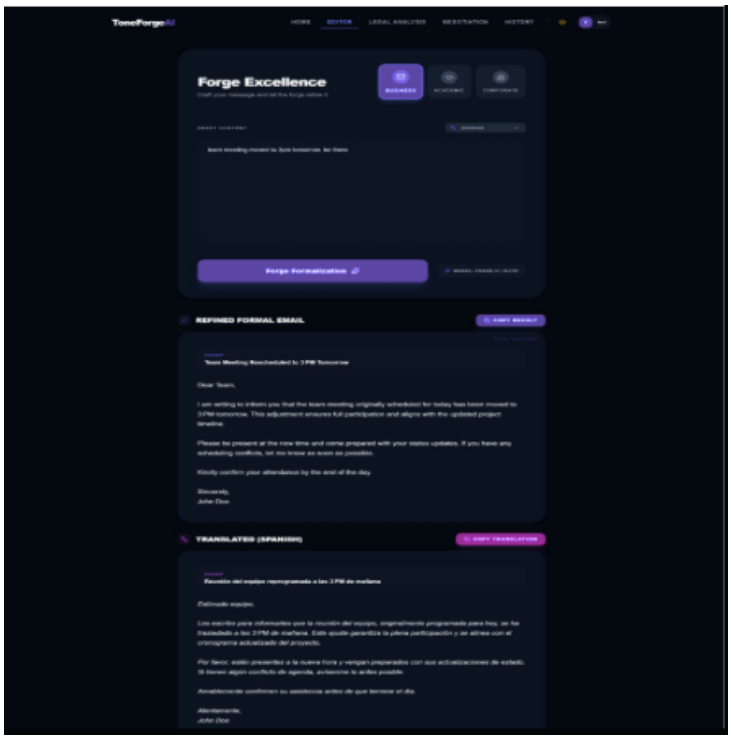


Figure 6.2: Email Formalization: Transforming Raw Input into Structured Business, Academic, and Corporate Registers

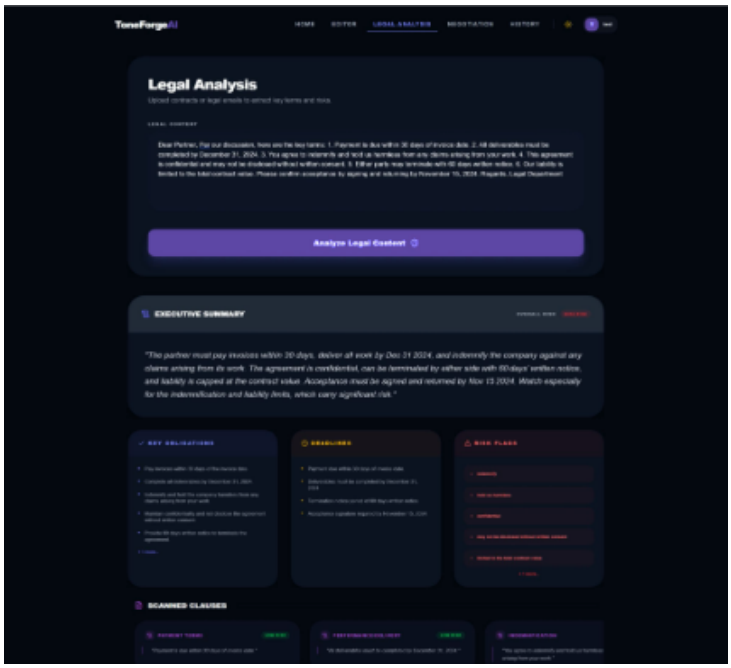


Figure 6.3: Legal Analysis Interface: Visualizing Risk Flags and Extracted Obligations

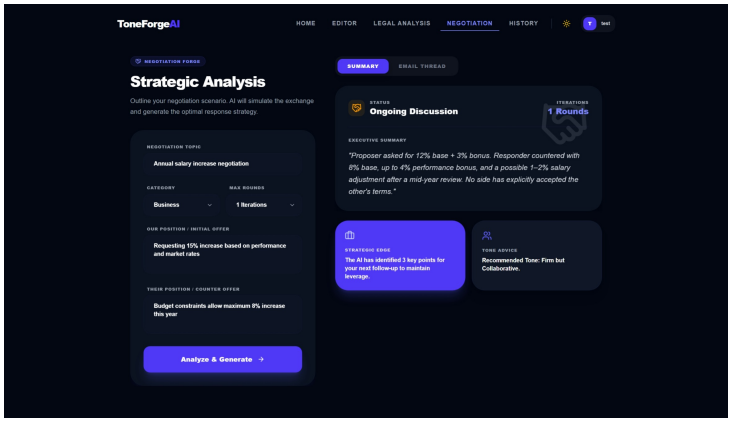


Figure 6.4: Multi-Agent Negotiation Dashboard: Simulating Iterative Communication Threads

Chapter 7

Conclusion and Future Scope

7.1 Challenges and Limitations

During the development of ToneForge, several technical hurdles were encountered and addressed:

- **Cyclic Agent State Management:** Managing the state within LangGraph required careful design to prevent infinite loops. Terminating the negotiation simulation required robust logic in the Evaluator node to definitively recognize agreement conditions, alongside a hard `max_rounds` parameter to act as a fail-safe.
- **Strict JSON Formatting from LLMs:** Open-source LLMs frequently prepend conversational filler to their output. To ensure the Node.js backend did not crash during JSON parsing, prompt escaping and Pydantic validation layers were heavily enforced at the FastAPI boundary to strip markdown and guarantee purely structured output.
- **Stateless Authorization Limits:** Because JWTs are stateless, implementing instant token revocation upon logout requires a server-side denylist, which adds database overhead. The current implementation mitigates this by relying on short-lived client-side token expiration.

7.2 Conclusion

The ToneForge project successfully demonstrates a highly modular, distributed architecture for deploying advanced AI workflows within a conventional web application. By decoupling the MERN stack web infrastructure from the Python-based AI microservice, the system achieves strong separation of concerns and independent scalability at each tier. The Node.js gateway ensures secure, persistent user management, while the LangGraph and FastAPI backend provides deterministic, schema-compliant text transformation. The successful implementation of cyclic multi-agent negotiation and structured legal parsing validates the hypothesis that current open-source LLM tooling can reliably extract and reformat operational meaning from unstructured human correspondence.

7.3 Future Scope

Several directions are identified for extending and enhancing the platform:

- **Asynchronous Task Queues:** Transitioning Node.js-to-Python communication from synchronous HTTP requests to a message broker (e.g., RabbitMQ with Celery workers) would better accommodate high-latency generation tasks and eliminate the risk of client-side timeouts for complex negotiation runs.
- **Retrieval-Augmented Generation (RAG):** Enabling users to upload PDF contracts and leveraging a vector database to supply the LLM with contextually relevant clause precedents would substantially improve the accuracy and depth of the legal parsing feature.
- **Custom Model Deployment:** Replacing the Groq-hosted inference endpoint with privately hosted, fine-tuned quantized models would enhance data privacy for organizations handling sensitive corporate communications, and reduce per-inference operational costs at scale.

Bibliography

[1] S. Bubeck et al., "Sparks of Artificial General Intelligence: Early Experiments with GPT-4," *arXiv preprint arXiv:2303.12528*, 2023.

[2] H. Chase et al., *LangChain: Building Applications with LLMs through Composability*. GitHub Repository, 2023. [Online]. Available: `https://github.com/langchain-ai/langchain`

[3] LangChain Inc., *LangGraph Documentation*. 2024. [Online]. Available: `https://langchain-ai.github.io/langgraph/`

[4] S. Ramírez, *FastAPI Documentation*. 2024. [Online]. Available: `https://fastapi.tiangolo.com`

[5] Pydantic Services Inc., *Pydantic v2 Documentation*. 2024. [Online]. Available: `https://docs.pydantic.dev`

[6] Node.js Foundation, *Node.js Documentation*. 2024. [Online]. Available: `https://nodejs.org/en/docs`

[7] OpenAPI Initiative, *OpenAPI Specification 3.0*. 2024. [Online]. Available: `https://spec.openapis.org/oas/v3.0.3`

[8] J. S. Park et al., "Generative Agents: Interactive Simulacra of Human Behavior," *arXiv preprint arXiv:2304.03442*, 2023.

Individual Contribution Report

ToneForge: Distributed AI System for Email Formalization, Multi-Agent Negotiation, and Legal Parsing

Kush Singh, Roll No. 23052247

Abstract: ToneForge is a distributed MERN-stack and Python microservice application for executing advanced LLM workflows. It delivers user-authenticated text formalization, LangGraph-driven multi-agent negotiation simulation, and structured legal email parsing.

Individual Contribution and Findings: Designed and implemented the agentic components of the LLM workflow, including the multi-agent negotiation simulation (Proposer, Responder, and Evaluator graph nodes) and the structured legal email analysis pipeline for risk assessment and clause extraction. Also contributed to the deployment configuration of the AI microservice.

Contribution to Project Report: Authored the sections covering the multi-agent system architecture, LangGraph state graph design, legal parsing schema definitions, and the integration methodology between LangGraph and FastAPI.

Contribution to Project Presentation: Demonstrated the multi-agent negotiation simulation and the structured legal email analysis feature during the project presentation, including a live walkthrough of agent-level outputs and risk classification results.

Supervisor Signature: **Student Signature:**

Dr. Ajit Kumar Pasayat Kush Singh

Individual Contribution Report

ToneForge: Distributed AI System for Email Formalization, Multi-Agent Negotiation, and Legal Parsing

Arup Mishra, Roll No. 23052229

Abstract: ToneForge is a distributed MERN-stack and Python microservice application for executing advanced LLM workflows. It delivers user-authenticated text formalization, LangGraph-driven multi-agent negotiation simulation, and structured legal email parsing.

Individual Contribution and Findings: Implemented the complete backend web service, including MongoDB schema design, Express route definitions, JWT-based authentication middleware, and database integration logic. Additionally developed the React frontend interfaces for the Negotiation, Signup, and Login pages.

Contribution to Project Report: Authored the sections covering the RESTful API service map, JWT authentication and authorization flows, and the Mongoose data schema specifications for user and history documents.

Contribution to Project Presentation: Demonstrated the backend deployment workflow, MongoDB history tracking, and the secure login and authentication flows during the project presentation.

Supervisor Signature: **Student Signature:**

Dr. Ajit Kumar Pasayat Arup Mishra

Individual Contribution Report

Shreya Banerjee, Roll No. 23052267

Abstract: ToneForge is a distributed MERN-stack and Python microservice application for executing advanced LLM workflows. It delivers user-authenticated text formalization, LangGraph-driven multi-agent negotiation simulation, and structured legal email parsing.

Individual Contribution and Findings: Developed the landing page, email editor, legal analysis, and history pages using React, implementing the full user-facing interface for AI email generation and legal risk analysis, and integrating each view with the backend REST API.

Contribution to Project Report: Authored the sections covering the frontend framework architecture, component-based UI structure, and client-side routing strategies using React Router.

Contribution to Project Presentation: Navigated the user interface elements during the presentation, highlighting the design and interaction model of the dashboard, the email editor, and the historical data retrieval view.

Supervisor Signature: **Student Signature:**

Dr. Ajit Kumar Pasayat Shreya Banerjee

Individual Contribution Report

ToneForge: Distributed AI System for Email Formalization, Multi-Agent Negotiation, and Legal Parsing

Devayush Kar, Roll No. 23052239

Abstract: ToneForge is a distributed MERN-stack and Python microservice application for executing advanced LLM workflows. It delivers user-authenticated text formalization, LangGraph-driven multi-agent negotiation simulation, and structured legal email parsing.

Individual Contribution and Findings: Implemented the core LLM workflow features including the formalization pipelines for business, academic, and corporate registers, the smart reply generation functionality, and the deployment of the AI microservice to Hugging Face Spaces.

Contribution to Project Report: Authored the sections covering the core AI processing pipelines, structured output formatting methodologies using Pydantic, and domain-specific email transformation logic.

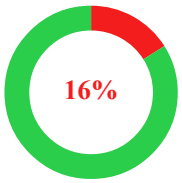
Contribution to Project Presentation: Presented the core email automation features, illustrating the end-to-end transformation from informal user input to structured, domain-appropriate professional output via register-specific prompt templates.

Supervisor Signature: **Student Signature:**

Dr. Ajit Kumar Pasayat Devayush Kar

SmallSEOTools

Plagiarism Detection Report by SmallSEOTOOLS



Plagiarism	16%	Partial Match	0%
Exact Match	16%	Unique	84%

Scan details

Total Words	Total Characters	Plagiarized Sentences	Unique Sentences
1020	5191	7.36	38.64 (84%)

Plagiarism Results: (7)

#1 2% Similar https://www.iust.ac.in/departmentcontent/DOELL/d…

In Partial Fulfillment of the Requirements for the Award of the

#2 2% Similar https://www.studocu.com/in/document/hi-tech-instit…

out by them in partial fulfillment of the requirements for the award of the Degree of

#3 2% Similar https://open.spotify.com/track/3jVtlIWS5CFFWLQn…

. 2

#4 4% Similar https://www.netflix.com/title/80074220

. 3

. 3

#5 2% Similar https://en.wikipedia.org/wiki/5_(disambiguation)

Powered by SmallSEOTools Plagiarism Checker Blog About Us Contact Us

. 5

#6 2% Similar https://www.youtube.com/watch?v=Lw-D8ThqWaE

. 6

#7 2% Similar https://en.wikipedia.org/wiki/11_(number)

. 11